

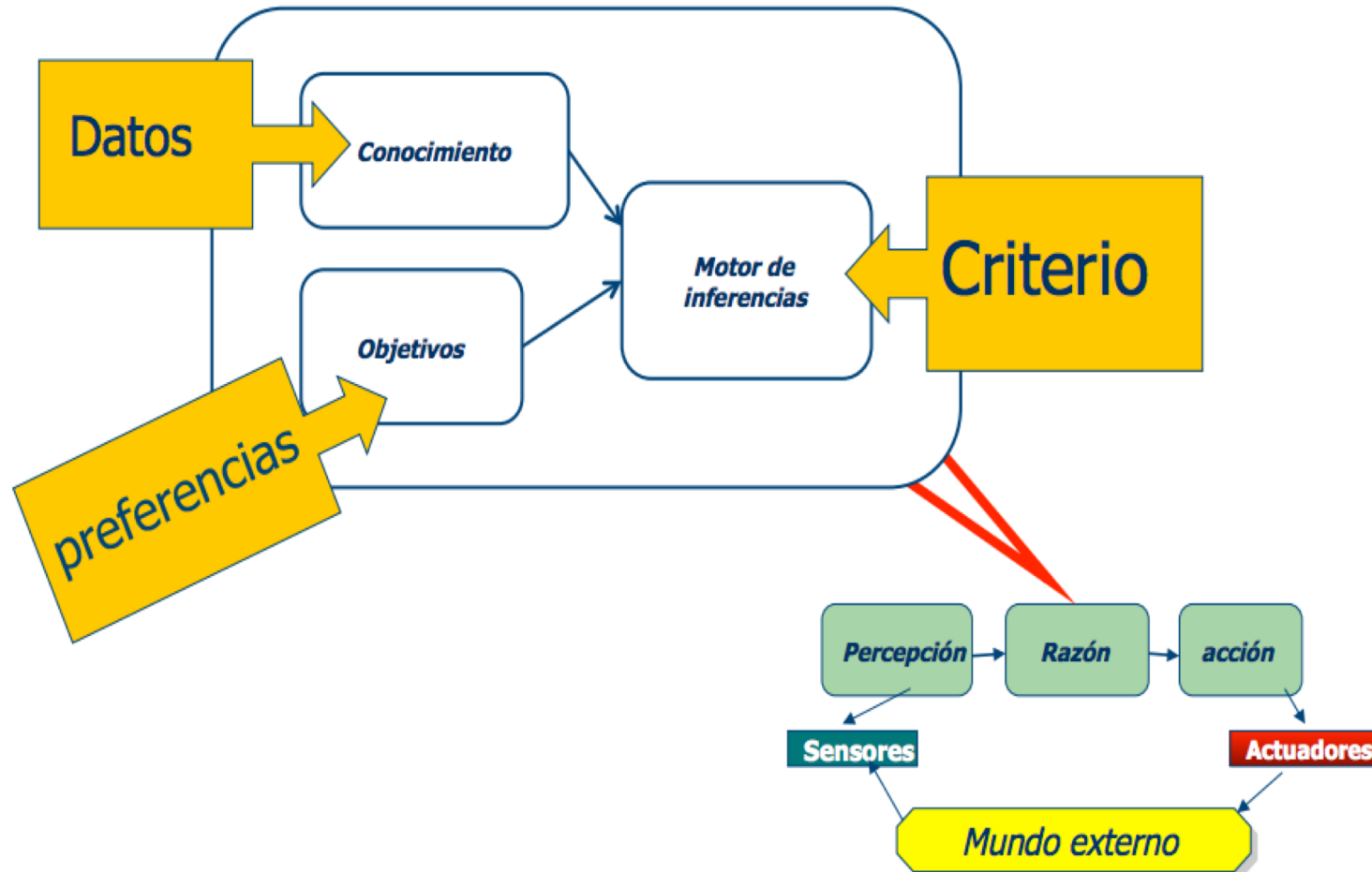
Sistema de Ayuda toma de decisiones

Introducción y Conceptos Básicos

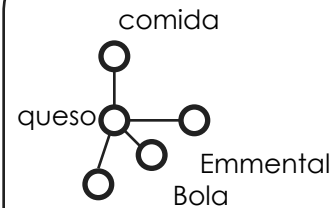
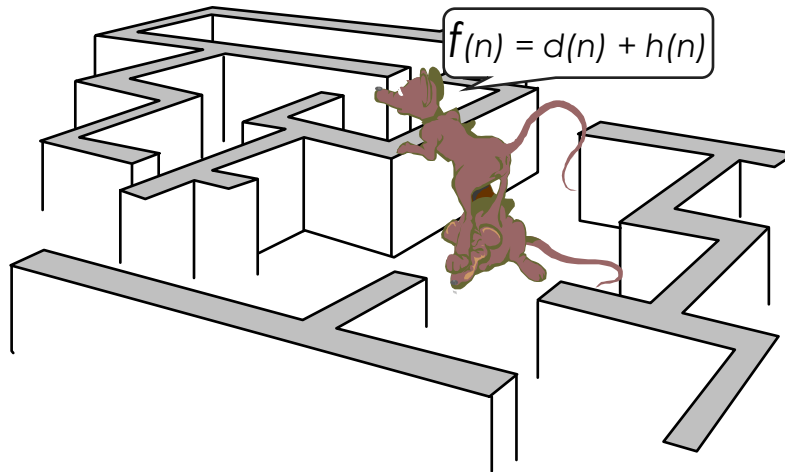
Luis Fernando Castillo Ossa

Sistemas Inteligentes 1

Cómo decidimos Racionalmente



Parte I. Representación del conocimiento

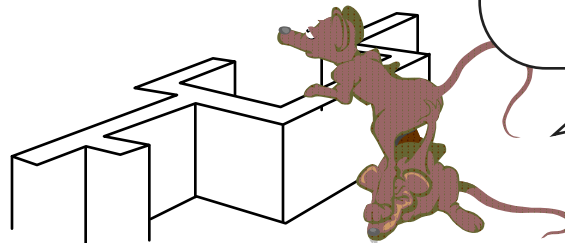


regla 101

Si huele a queso por aqui y
no veo trampas
entonces merodear cerca

regla 103

Si ya he pasado por aqui
entonces intentar otra alternativa



Índice

1. Sistemas de Producción/
Lenguajes Basados en reglas
2. Arquitectura de los Lenguajes
Basados en Reglas
3. Representación Memoria de
trabajo y producción
4. Proceso de Reconocimiento
 - 4.1 Red de Inferencia
 - 4.2 Sistemas de reconocimiento de
patrones
5. El proceso de razonamiento
 - 5.1 Encadenamiento progresivo
 - 5.2 Encadenamiento regresivo
 - 5.3 Encadenamiento/Razonamiento
 - 5.4 Estrategias de control
6. Ventajas y desventajas de los
LBR
7. Redes y Frames

1. Sistemas de Producción / Lenguajes basados en reglas

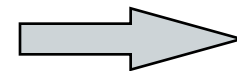
- Un Sistema de Producción ofrece un mecanismo de control dirigido por patrones para la resolución de problemas

- Representación del conocimiento experto

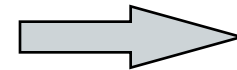
Se puede apreciar que las nubes están oscuras y está levantándose viento,

y cuando se dan estos hechos siempre llueve copiosamente.

```
(defrule lluvia
  (nubes (color oscuro))
  (atmosfera (viento moderado))
=>
  (assert (fenomeno (lluvia))))
```



Patrón



Acción

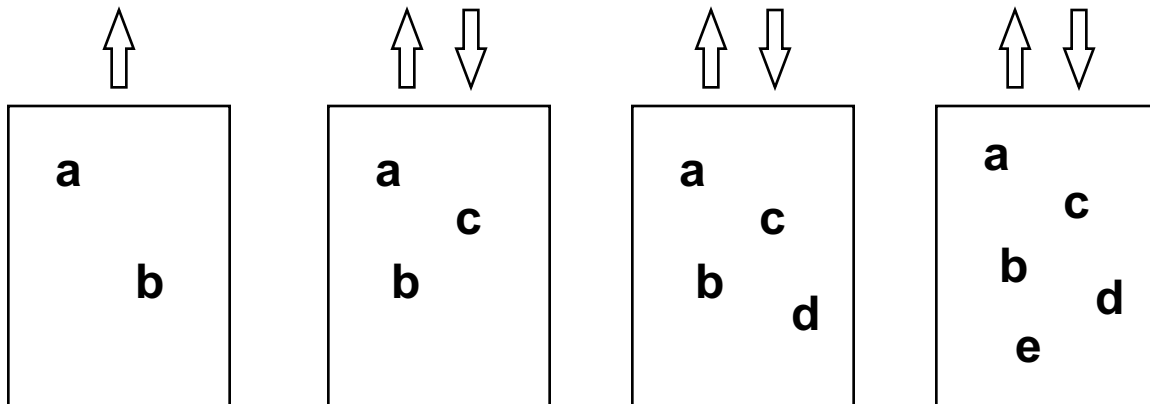
Inferencia en lenguajes basados en reglas

- Una regla no aporta mucho, pero un conjunto de reglas pueden formar una cadena capaz de alcanzar una conclusión significativa.

Regla 1: SI la temperatura ambiente es de 32°
ENTONCES el tiempo es caluroso
Regla 2: SI la humedad relativa es mayor que el 65%
ENTONCES la atmosfera está húmeda
Regla 3: SI el tiempo es caluroso y la atmósfera está húmeda
ENTONCES se va a formar una tormenta

Reglas

Intérprete



Memoria de Trabajo

a = La temperatura ambiente es de 32°
b = La humedad relativa es mayor que el 65%
c = El tiempo es caluroso
d = La atmósfera está húmeda
e = se va a formar una tormenta

Cuando utilizar un lenguaje basado en reglas

■ Problema :

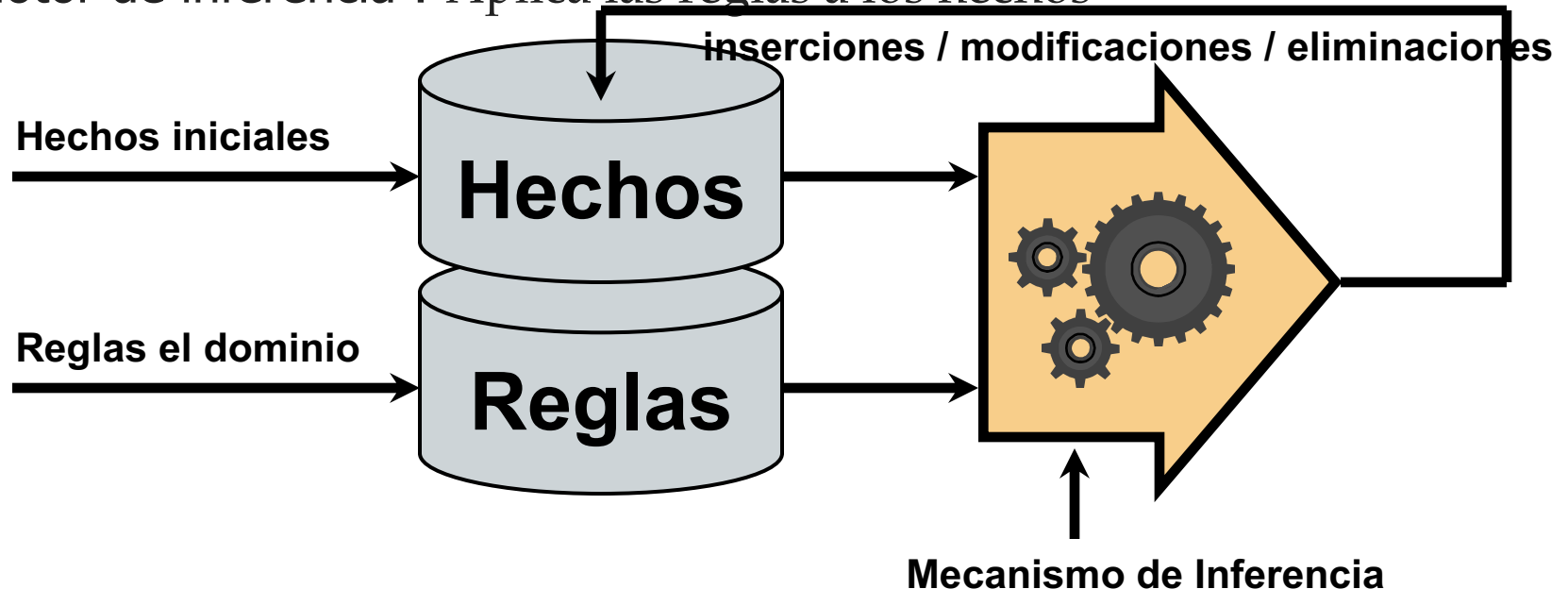
- No siempre es fácil (e incluso posible) obtener la manera de resolver un problema mediante una solución algorítmica
- Se busca simular el razonamiento humano en dominios en los que el conocimiento es “evolutivo” y en los cuales no existe un método determinista seguro:
 - Ejemplos en las finanzas, la economía, las ciencias sociales
 - Ejemplos en la medicina (diagnóstico médico), en el mantenimiento
 - Ejemplos de demostración automática de teoremas, ...

■ Método :

- Aislar y modelar un subconjunto del mundo real
- Modelar el problema en términos de hechos iniciales o de objetivos a conseguir.
- Modelar los patrones y las acciones simulando el razonamiento humano.

Arquitectura de los lenguajes basados en reglas

- Base de hechos : Contiene hechos iniciales, más los deducidos
- Base de Reglas : Contienen las reglas que explotan los hechos
- Motor de inferencia : Aplica las reglas a los hechos



Dos partes de un Sistema Basado en Reglas

- Parte declarativa :
 - **Hechos** : conocimiento declarativo (información) sobre el mundo real
 - **Reglas** : conocimiento declarativo de la gestión de la base de hechos de gestión de la base de hechos
 - En lógica *monótona* : únicamente se permiten añadir hechos
 - En lógica *non-monótona* : inserciones, modificaciones y eliminación de hechos
 - **Meta-reglas** : conocimiento declarativo sobre el empleo de las reglas
- Parte algorítmica/imperativa :
 - **Motor de inferencia** : Software que efectúa los *razonamientos* sobre el conocimiento declarativo disponible :
 - Aplica las reglas de la memoria de producción a los hechos en memoria de trabajo
 - Se basa en uno o varios esquemas de razonamiento (ex : deducción)
 - Se puede consultar la traza del proceso de razonamiento:
 - Durante las fases de diseño y depuración
 - Para obtener explicaciones sobre la solución

Ejemplos de deducción

Modus Ponens

Modus Tollens

Inferencias sin variables :

(hombre Sócrates) es cierto

(hombre Sócrates) $\rightarrow$ (mortal Sócrates)

(mortal Sócrates) es cierto

(mortal Agustín) es falso

(hombre Agustín) $\rightarrow$ (mortal Agustín)

(hombre Agustín) es falso

Inferencias con variables :

(hombre Sócrates) es cierto

(hombre ?x) $\rightarrow$ (mortal ?x)

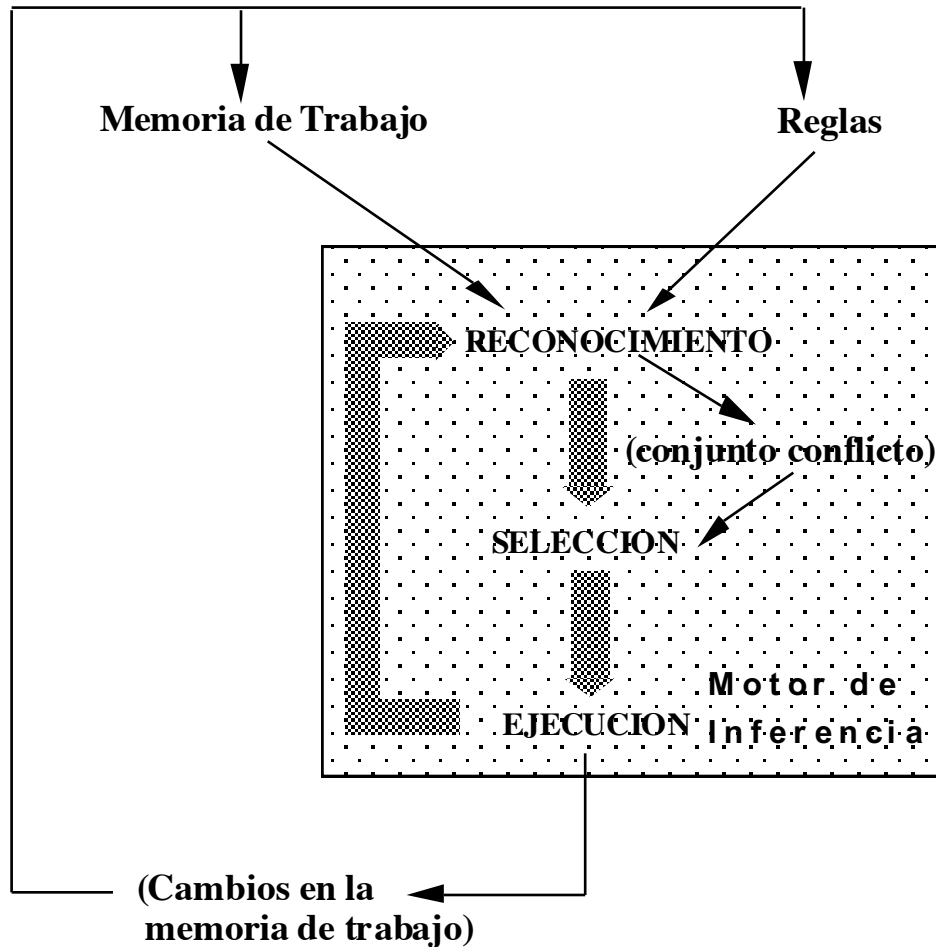
(mortal Sócrates) es cierto

(mortal Agustín) es falso

(hombre ?x) $\rightarrow$ (mortal ?x)

(hombre Agustín) es falso

Motor de Inferencia (OPS5/CLIPS)



- La interpretación de las reglas conlleva los siguientes pasos básicos:
 - **Reconocimiento:** Comparación de los patrones en las reglas con los elementos de la memoria de trabajo.
 - **Resolución de conflictos:** Se elige una regla entre las satisfechas por la memoria de trabajo y se ejecuta su parte ENTONCES.
 - **Ejecución:** La ejecución de las reglas da lugar a cambios en la memoria de trabajo. (También podrían añadirse nuevas reglas)
- La estrategia de control especifica La forma de resolver conflictos

Reglas

Regla 1: SI mes = mayo ... octubre
ENTONCES estación = húmeda

Regla 2: SI mes = noviembre ... abril
ENTONCES estación = seca

Regla 3: SI precipitaciones = ninguna
estación = seca
ENTONCES cambio = bajo

Regla 4: SI precipitaciones = ninguna
estación = húmeda
ENTONCES cambio = ninguno

Regla 5: SI precipitaciones = moderada
ENTONCES cambio = ninguno

Regla 6: SI precipitaciones = alta
ENTONCES cambio = alto

Regla 7: SI nivel = bajo
ENTONCES alerta = no, evacuación = no

Regla 8: SI cambio = ninguno | bajo
nivel = normal | bajo
ENTONCES alerta = no, evacuación = no

Regla 9: SI cambio= alto, nivel= normal
lluvia = intensa
ENTONCES alerta = si (FC 0.4)
evacuación = no

Regla 10: SI cambio=alto, nivel= normal
lluvia = ligera
ENTONCES alerta = no, evacuación = no

Regla 11: SI cambio= alto, nivel= alto
lluvia = ninguna|ligera
ENTONCES alerta = si (FC 0.5)
evacuación = si (FC 0.2)

Regla 12: SI cambio= alto,nivel= alto
lluvia = intensa
ENTONCES alerta = si
evacuación = si (FC 0.8)

Regla 13: SI altura < 1
ENTONCES nivel = bajo

Regla 14: SI altura >= 1 and <=2
ENTONCES nivel = normal

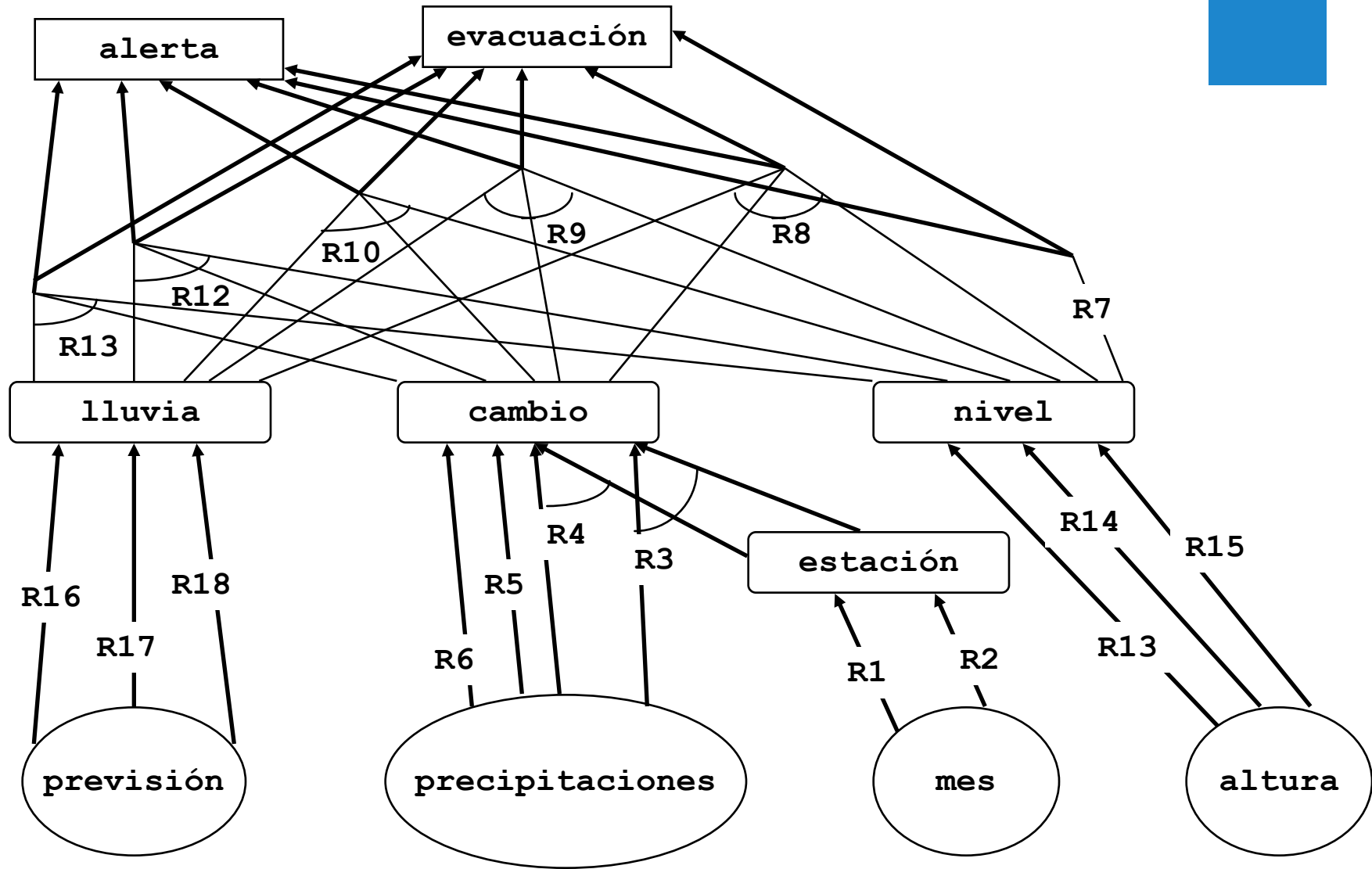
Regla 15: SI altura >2
ENTONCES nivel = alto

Regla 16: SI previsión = soleado
ENTONCES lluvia = ninguna

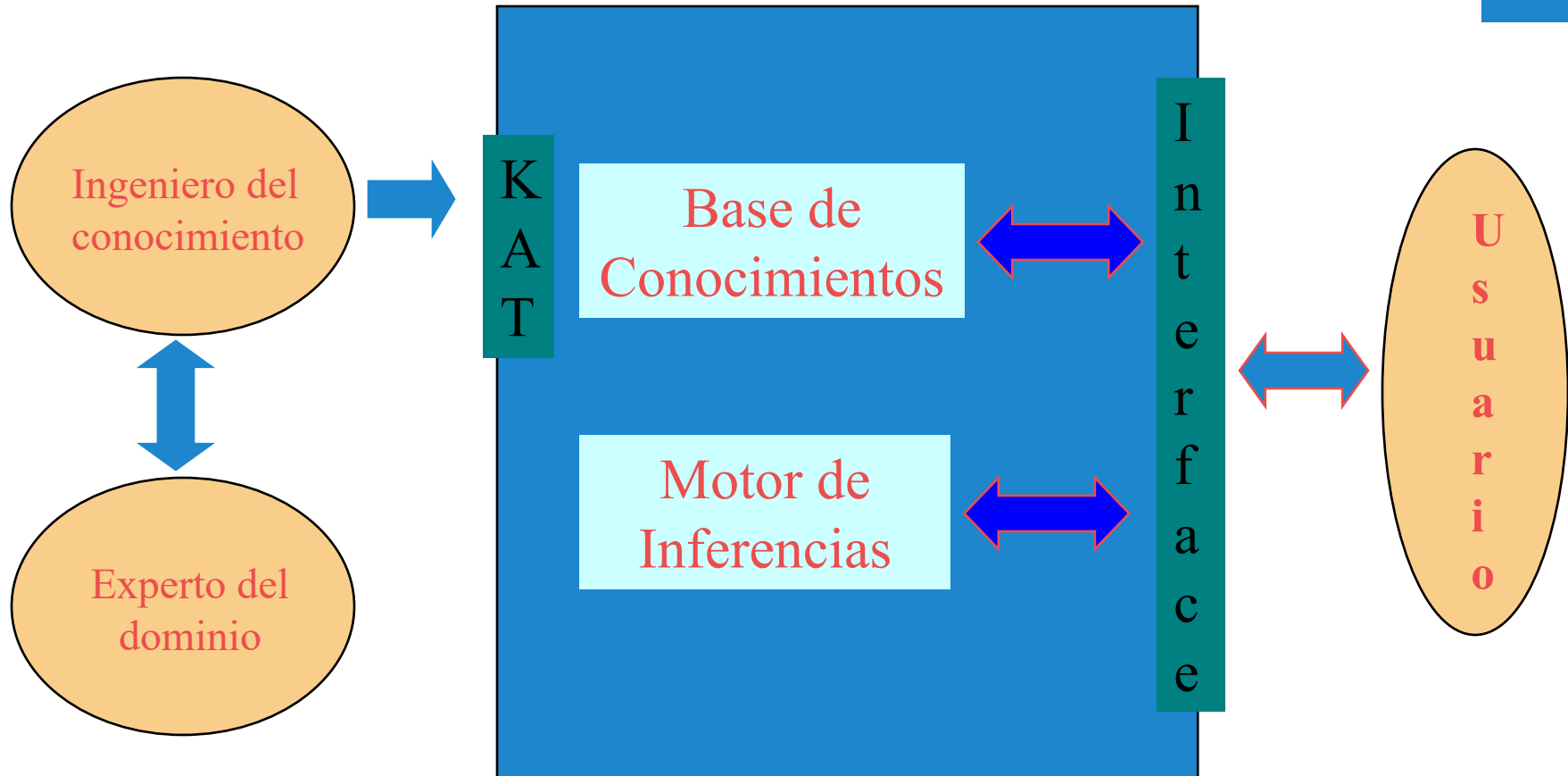
Regla 17: SI previsión = nublado
ENTONCES lluvia = ligera

Regla 18: SI previsión = tormentoso
ENTONCES lluvia = intensa

Red de Inferencia



Estructura básica de un SE.

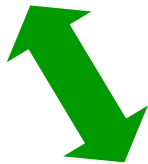
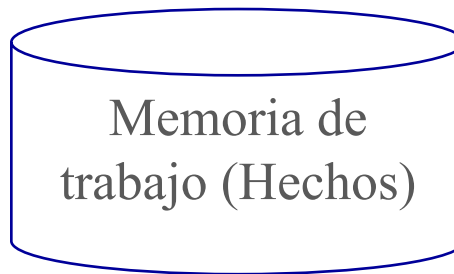
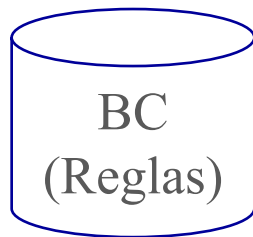


SE basados en reglas de producción

Sistemas de producción



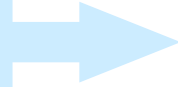
Newell y Simon (1972): Al resolver problemas, las personas utilizan su *memoria a largo plazo* (permanente) que aplican a situaciones actuales contenidas en su *memoria a corto plazo* (transitoria). Esto puede generar modificaciones en la última.



Motor de Inferencias

SE basados en reglas de producción

MOTOR DE
INFERENCIAS



Dos formas de funcionamiento.



Inductivo: A partir de un objetivo intenta verificar los hechos que los sostienen



BACKWARD
CHAINING



Deductivo: A partir de los hechos disponibles infiere todas las conclusiones posibles



FORWARD
CHAINING

Encadenamiento hacia atrás - Backward Chaining.

BH := CONOCIMIENTO INICIAL (HECHOS).

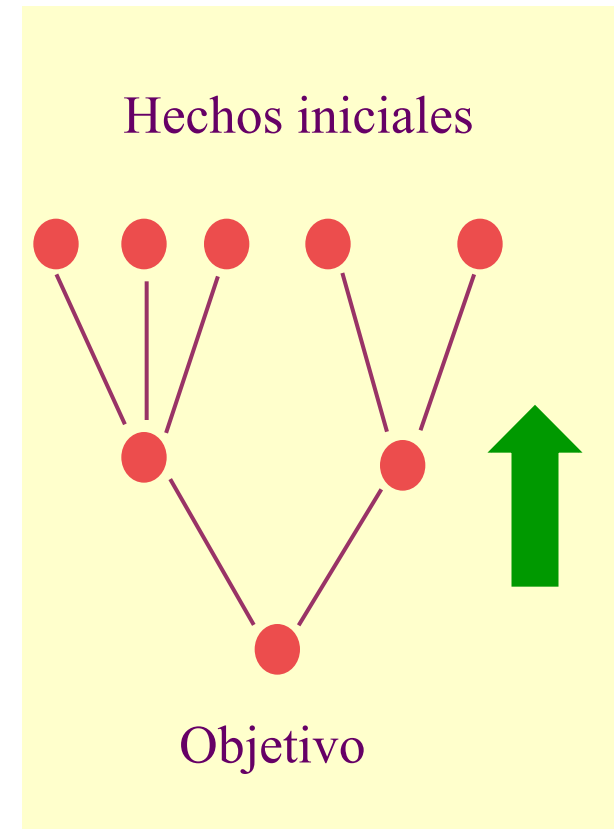
HASTA OBJETIVO O SIN REGLAS PARA DISPARAR.

REPITA

(1) ENCONTRAR K CONJUNTO DE REGLAS, CUYAS CONCLUSIONES PUEDEN UNIFICARSE CON LA HIPÓTESIS (CONJUNTO DE CONFLICTO).

(2) ELEGIR R DE K SEGÚN ESTRATEGIA DE SOLUCIÓN DE CONFLICTOS (POSIBLE BACKTRACKING).

(3) SI LA PREMISA DE R NO ESTÁ EN BH, TOMARLA COMO SUBOBJETIVO.



Backward Chaining: Ciclo base de un motor inductivo.

DETECCIÓN:

- ✓ SI EL OBJETIVO ES CONOCIDO ÉXITO.
- ✓ SINO, TOMAR LAS REGLAS QUE LO CONCLUYEN (CC).

ELECCIÓN:

- ✓ DECIDIR QUE REGLA APLICAR (RC)

APLICACIÓN:

- ✓ REEMPLAZAR EL OBJETIVO POR LA CONJUNCIÓN DE LAS CONDICIONES DE LA PREMISA ELEGIDA.



Ejemplo Backward Chaining: Primer paso

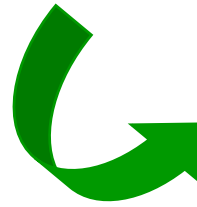
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ OBJETIVO: v

➤ $v \in BH$?

➤ $v \notin BH$

➤ SIGUE



BH: $p \ q \ r$

Ejemplo Backward Chaining: Segundo paso

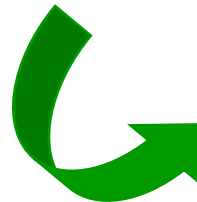
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ MATCHING CON v

➤ $CC = \{ R4 \}$

➤ $s \in BH?$

➤ $s \notin BH$

➤ s SUBOBJETIVO

➤ SIGUE



BH: $p \ q \ r$



Ejemplo Backward Chaining: Tercer paso

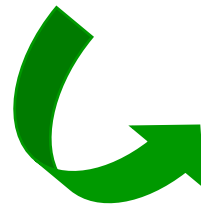
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



BH: $p \ q \ r$

- MATCHING CON s
- $CC = \{ R1 \}$
- $p \in BH$? SI.
- $q \in BH$? SI.
- DISPARA R1.
- $BH \leftarrow s$

Ejemplo Backward Chaining: 4º paso

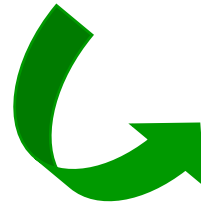
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ $CC = \{ R4 \}$

➤ $r \in BH?$ SI.

➤ DISPARA R4.

➤ $BH \leftarrow v$



BH: p q r s

Ejemplo Backward Chaining: 5º paso

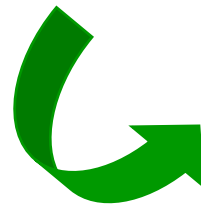
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ OBJETIVO OK.

➤ FIN



BH: p q r s v

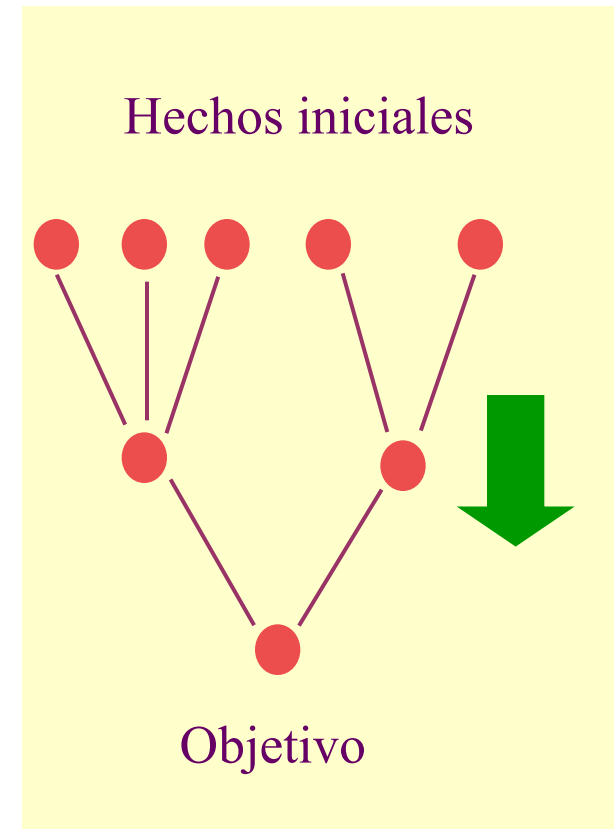
Encadenamiento hacia adelante - Forward Chaining.

BH := CONOCIMIENTO INICIAL (HECHOS).

HASTA OBJETIVO O SIN REGLAS PARA DISPARAR.

REPITA:

- (1) ENCONTRAR K CONJUNTO DE REGLAS CUYAS PREMISAS CUMPLEN CON BH (CONJUNTO DE CONFLICTO-CC).
- (2) ELEGIR R DE K SEGÚN ESTRATEGIA DE SOLUCIÓN DE CONFLICTOS (RC).
- (3) DISPARAR R Y ACTUALIZAR BH. (RECORDAR R).





Forward Chaining: Ciclo base de un motor deductivo.



Ejemplo Forward Chaining: Primer paso

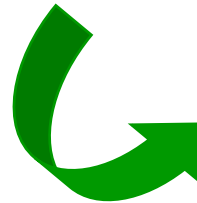
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ $CC = \{ R1, R2 \}$

➤ $R1 \leftarrow RC$

➤ DISPARA R1

➤ $BH \leftarrow s$

➤ R1 APLICADA



BH: $p \ q \ r$

Ejemplo Forward Chaining: 2º paso

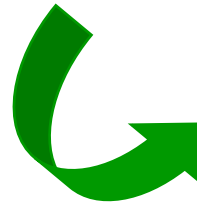
BASE DE REGLAS

R1: $p \wedge q \rightarrow s$

R2: $r \rightarrow t$

R3: $s \wedge t \rightarrow u$

R4: $s \wedge r \rightarrow v$



➤ $CC = \{ R2, R4 \}$

➤ $R2 \leftarrow RC$

➤ DISPARA R2

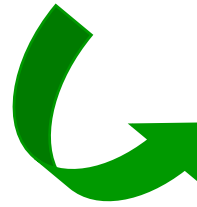
➤ $BH \leftarrow t$

➤ R2 APLICADA



BH: $p \ q \ r \ s$

Ejemplo Forward Chaining: 3er paso

BASE DE REGLASR1: $(p \wedge q \rightarrow s)$ R2: $(r \rightarrow t)$ R3: $s \wedge t \rightarrow u$ R4: $s \wedge r \rightarrow v$ ➤ $CC = \{ R3, R4 \}$ ➤ $R3 \leftarrow RC$

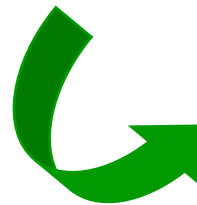
➤ DISPARA R3

➤ $BH \leftarrow u$

➤ R3 APLICADA

BH: p q r s t

Ejemplo Forward Chaining: 4º paso

BASE DE REGLAS(R1: $p \wedge q \rightarrow s$)(R2: $r \rightarrow t$)(R3: $s \wedge t \rightarrow u$)R4: $s \wedge r \rightarrow v$ ➤ $CC = \{R4\}$ ➤ $R4 \leftarrow RC$

➤ DISPARA R4

➤ $BH \leftarrow v$

➤ R4 APLICADA

BH: p q r s t u

Ejemplo Forward Chaining: 5º paso

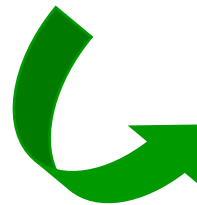
BASE DE REGLAS

(R1: $p \wedge q \rightarrow s$)

(R2: $r \rightarrow t$)

(R3: $s \wedge t \rightarrow u$)

(R4: $s \wedge r \rightarrow v$)



➤ CC = { }

➤ FIN



BH: p q r s t u v

Sistemas de Producción.

Patrones con variables

1. Variables en las reglas CLIPS

- variable ::= ?<nombre>

```
CLIPS> (deftemplate personaje (slot nombre) (slot ojos)
                                             (slot pelo))
```

```
CLIPS> (defrule busca-ojos-azules
        (personaje (nombre ?nombre) (ojos azules))
        =>
        (printout t ?nombre " tiene los ojos azules."
        crlf))
```

```
CLIPS> (defacts gente
        (personaje (nombre Juan) (ojos azules) (pelo castagno))
        (personaje (nombre Luis) (ojos verdes) (pelo rojo))
        (personaje (nombre Pedro) (ojos azules) (pelo rubio))
        (personaje (nombre Maria) (ojos castagnos) (pelo negro)))
```

```
CLIPS> (reset)
```

```
CLIPS> (run)
```

Pedro tiene los ojos azules.

Juan tiene los ojos azules.

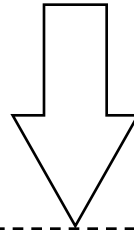
Reconocimiento de patrones

(defrule COMER
 (HAMBRIENTO ?PERSONA)
 (COMESTIBLE ?ALIMENTO)
 =>
 (assert (Come ?PERSONA el ?ALIMENTO)))

BASE de REGLAS

H1: (HAMBRIENTO PEDRO)
 H2: (HAMBRIENTO PABLO)
 H3: (COMESTIBLE MANZANA)
 H4: (COMESTIBLE MELOCOTON)
 H5: (COMESTIBLE PERA)

BASE de HECHOS



- I1: (?PERSONA = PEDRO, ?ALIMENTO = MANZANA) (H1,H3)
- I2: (?PERSONA = PEDRO, ?ALIMENTO = MELOCOTON (H1,H4)
- I3: (?PERSONA = PEDRO, ?ALIMENTO = PERA (H1,H5)
- I4: (?PERSONA = PABLO, ?ALIMENTO = MANZANA (H2,H3))
- I5: (?PERSONA = PABLO, ?ALIMENTO = MELOCOTON (H2 ,H4))
- I6 :(?PERSONA = PABLO, ?ALIMENTO = PERA (H2 ,H5))

Variable que se repiten en distintos patrones



2. Variables en las reglas CLIPS: Restricción por ligaduras consistentes

```
CLIPS> (undefrule *)
CLIPS>
(deftemplate busca (slot ojos))
CLIPS>
(defrule busca-ojos
  (busca (ojos ?color-ojos))
  (personaje (nombre ?nombre) (ojos ?color-ojos))
  =>
  (printout t ?nombre " tiene los ojos " ?color-ojos ".")
  crlf))
CLIPS> (reset)
CLIPS> (assert (busca (ojos azules)))
<Fact-5>
CLIPS> (run)
Pedro tiene los ojos azul.
Juan tiene los ojos azul.
```

Funciones avanzadas para la correspondencia

■ Ligar un hecho a una variable

```
CLIPS> (deftemplate persona (slot nombre) (slot direccion))
CLIPS> (deftemplate cambio (slot nombre) (slot direccion))
CLIPS> (defrule procesa-informacion-cambios
  ?h1 <- (cambio (nombre ?nombre) (direccion ?direccion))
  ?h2 <- (persona (nombre ?nombre))
  =>
    (retract ?h1)
    (modify ?h2 (direccion ?direccion)))
CLIPS> (deffacts ejemplo
  (persona (nombre "Pato Donald") (direccion "Disneylandia"))
  (cambio (nombre "Pato Donald") (direccion "Port Aventura")))
CLIPS> (reset)
CLIPS> (run)
CLIPS> (facts)
f-0 (initial-fact)
f-3 (persona (nombre "Pato Donald") (direccion "Port Aventura"))
For a total of 2 facts.
```

Comodines

- Variable que no liga valor y reconoce cualquier valor: ?

```
CLIPS> (clear)
```

```
CLIPS> (deftemplate persona (multislot nombre) (slot  
dni))
```

```
CLIPS>
```

```
(deffacts ejemplo
```

```
  (persona (nombre Jose L. Perez) (dni 22454322))
```

```
  (persona (nombre Juan Gomez) (dni 23443325)))
```

```
CLIPS>
```

```
(defrule imprime-dni
```

```
  (persona (nombre ? ? ?Apellido) (dni ?dni))
```

```
=>
```

```
  (printout t ?dni " " ?Apellido crlf))
```

```
CLIPS> (reset)
```

```
CLIPS> (run)
```

```
22454322 Perez
```

Comodines para varios valores

- Comodín que reconoce cero o más valores de un atributo multivalor: `$?`

```
CLIPS>
```

```
(defrule imprime-dni  
  (persona (nombre $?nombre ?Apellido) (dni ?dni))  
=>  
  (printout t ?dni " " ?nombre " " ?Apellido crlf))
```

```
CLIPS> (reset)
```

```
CLIPS> (run)
```

```
23443325 (Juan) Gomez
```

```
22454322 (Jose L.) Perez
```


Restricciones sobre el valor de un atributo

■ Operador `~`

```
(persona (nombre ?nombre) (pelo ~rubio))
```

■ Operador `|`

```
(persona (nombre ?nombre) (pelo castagno | pelirojo))
```

■ Operador `&` (En combinación con los anteriores para ligar valor a una variable)

```
(defrule pelo-castagno-o-rubio  
  (persona (nombre ?nombre) (pelo ?color&rubio|castagno))  
  =>  
  (printout t ?nombre " tiene el pelo " ?color crlf))
```

Restricciones sobre el valor

■ Predicados sobre el valor de un atributo

```
(defrule mayor-de-edad
  (persona (nombre $?nombre) (edad ?edad&:(> ?edad 18))
  =>
  (printout t ?nombre " es mayor de edad. Edad:" ?edad crlf))
```

■ Predicados sobre el valor de un atributo (igualdad)

```
(defrule mayor-de-edad
  (persona (nombre $?nombre) (edad =(+ 18 2))
  =>
  (printout t ?nombre
    " tiene dos años sobre la mayoría de edad." crlf))
```

■ Función Test

```
(test (> ?edad 18))
```

■ Función Bind (liga un valor no obtenido mediante reconocimiento a una variable)

```
(bind ?suma (+ ?a ?b))
```

Patrones en sistemas basados en reglas

- Utilización de implementaciones sofisticadas de algoritmos para reconocimiento de patrones para
 - ligar valores a variables,
 - restringir los valores que pueden tomar las variables
- Las relaciones entre las reglas y los hechos se determinan en tiempo de ejecución.
 - En lugar de parámetros se utilizan hechos con múltiples valores (tuplas objeto-atributos-valor), y patrones con variables que pueden reconocer diferentes hechos.
 - Los lenguajes basados en reglas con patrones permiten elaborar restricciones complejas para la identificación de hechos en la base de datos.

Evaluación de arquitecturas

- **Sistemas con reconocimiento de patrones:**
 - Aplicables a dominios con un gran número de soluciones, o donde no hay una relación clara entre las reglas y los hechos.
 - Son muy flexibles para expresar patrones.
 - La búsqueda de hechos que satisfacen las reglas es muy ineficiente.

- **Sistemas basados en redes de inferencia**
 - Aplicables a dominios donde están limitadas el número de soluciones.
 - Más fácilmente implementable.
 - Menos flexible.

El proceso de razonamiento (adelante y atrás)

- El proceso de razonamiento es una progresión desde un conjunto de datos hacia una solución, respuesta o conclusión.
- Dos situaciones posibles:
 - Hay pocos datos iniciales, y muchas soluciones conclusiones. Lo razonable es progresar desde los datos iniciales hasta una solución.
 - Hay muchos datos iniciales, pero solo unos pocos son relevantes
 - *Por ejemplo, cuando vamos al médico solo le contamos los síntomas anormales (dolor de cabeza, nauseas). El médico intentará probar hipótesis preguntando cuestiones adicionales.*
- Razonamiento dirigido por los datos o encadenamiento progresivo:
 - Comienza con todos los datos conocidos y progresa hasta la conclusión
- Razonamiento dirigido por los objetivos o encadenamiento regresivo:
 - Selecciona una conclusión posible e intenta probar su validez buscando evidencias que la soporten.

Ejemplo encadenamiento

REGLAS PARA IDENTIFICAR FRUTA

- Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana
- Regla 2: SI Forma = redonda u ovalada
Diámetro > 1.6 cm
ENTONCES claseFruta = planta
- Regla 3: SI Forma = redonda y
Diámetro < 1.6 cm
ENTONCES claseFruta = árbol
- Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso
- Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple
- Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía
- Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón
- Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque
- Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja
- Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza
- Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón
- Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana
- Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Intérprete con encadenamiento progresivo

- Pasos del intérprete
 1. Reconocimiento: Encuentra reglas aplicables y márcalas.
 2. Resolución de conflictos: Desactiva reglas que no añadan hechos nuevos.
 3. Acción: Ejecuta la acción de la regla **aplicable con menor número**. Si no hay reglas aplicables se detiene el intérprete.
 4. Reset: Vacía la lista de reglas aplicables y vuelve al paso 1.

- Si la memoria de trabajo tiene los siguientes hechos iniciales:

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

<i>Ciclo de</i>	<i>Reglas</i>	<i>regla</i>	<i>Hecho derivado</i>
<i>ejecución</i>	<i>aplicables</i>	<i>seleccionada</i>	

1	3, 4	3	claseFruta = árbol
2	3, 4	4	claseSemilla = hueso
3	3, 4, 10	10	Fruta = cereza
4	3, 4, 10	—	

Intérprete con encadenamiento progresivo

- Pasos del intérprete
 1. Reconocimiento: Encuentra reglas aplicables y márcalas.
 2. Resolución de conflictos: Desactiva reglas que no añadan hechos nuevos.
 3. Acción: Ejecuta la acción de la regla **aplicable con menor número**. Si no hay reglas aplicables se detiene el intérprete.
 4. Reset: Vacía la lista de reglas aplicables y vuelve al paso 1.
- Si la memoria de trabajo tiene los siguientes hechos iniciales:

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

<i>Ciclo de</i>	<i>Reglas</i>	<i>regla</i>	<i>Hecho derivado</i>
<i>ejecución</i>	<i>aplicables</i>	<i>seleccionada</i>	

1	3, 4	3	claseFruta = árbol
2	3, 4	4	claseSemilla = hueso
3	3, 4, 10	10	Fruta = cereza
4	3, 4, 10	—	

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

REGLAS PARA IDENTIFICAR FRUTA

Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana

Regla 2: SI Forma = redonda u ovalada
y Diametro > 1.6 cm
ENTONCES claseFruta = planta

Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol

Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso

Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple

Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía

Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón

Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque

Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja

Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza

Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón

Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana

Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Intérprete con encadenamiento progresivo

- Pasos del intérprete
 1. Reconocimiento: Encuentra reglas aplicables y márcalas.
 2. Resolución de conflictos: Desactiva reglas que no añadan hechos nuevos.
 3. Acción: Ejecuta la acción de la regla **aplicable con menor número**. Si no hay reglas aplicables se detiene el intérprete.
 4. Reset: Vacía la lista de reglas aplicables y vuelve al paso 1.
- Si la memoria de trabajo tiene los siguientes hechos iniciales:

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

<i>Ciclo de</i>	<i>Reglas</i>	<i>regla</i>	<i>Hecho derivado</i>
<i>ejecución</i>	<i>aplicables</i>	<i>seleccionada</i>	

1	3, 4	3	claseFruta = árbol
2	3, 4	4	claseSemilla = hueso
3	3, 4, 10	10	Fruta = cereza
4	3, 4, 10	—	

Diametro = 0.4 cm **forma = redonda** **Numsemillas = 1,** **color = rojo,**
claseFruta=arbol

REGLAS PARA IDENTIFICAR FRUTA

Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana

Regla 2: SI Forma = redonda u ovalada
y Diametro > 1.6 cm
ENTONCES claseFruta = planta

Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol

Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso

Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple

Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía

Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón

Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque

Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja

Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza

Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón

Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana

Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Intérprete con encadenamiento progresivo

- Pasos del intérprete
 1. Reconocimiento: Encuentra reglas aplicables y márcalas.
 2. Resolución de conflictos: Desactiva reglas que no añadan hechos nuevos.
 3. Acción: Ejecuta la acción de la regla **aplicable con menor número**. Si no hay reglas aplicables se detiene el intérprete.
 4. Reset: Vacía la lista de reglas aplicables y vuelve al paso 1.
- Si la memoria de trabajo tiene los siguientes hechos iniciales:

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

<i>Ciclo de</i>	<i>Reglas</i>	<i>regla</i>	<i>Hecho derivado</i>
<i>ejecución</i>	<i>aplicables</i>	<i>seleccionada</i>	

1	3, 4	3	claseFruta = árbol
2	3, 4	4	claseSemilla = hueso
3	3, 4, 10	10	Fruta = cereza
4	3, 4, 10	—	

Diametro = 0.4 cm **forma = redonda** **Numsemillas = 1** **color = rojo**
claseSemilla=hueso **clasefruta= arbol**

REGLAS PARA IDENTIFICAR FRUTA

Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana

Regla 2: SI Forma = redonda u ovalada
y Diametro > 1.6 cm
ENTONCES claseFruta = planta

Regla 3: SI **Forma = redonda** y
Diametro < 1.6 cm
ENTONCES **claseFruta = árbol**

Regla 4: SI **numSemillas = 1** y
ENTONCES **claseSemilla = hueso**

Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple

Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía

Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón

Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque

Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja

Regla 10: SI **claseFruta = árbol** y
Color = rojo y
claseSemilla = hueso
ENTONCES **Fruta = cereza**

Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón

Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana

Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Intérprete con encadenamiento progresivo

- Pasos del intérprete
 1. Reconocimiento: Encuentra reglas aplicables y márcalas.
 2. Resolución de conflictos: Desactiva reglas que no añadan hechos nuevos.
 3. Acción: Ejecuta la acción de la regla **aplicable con menor número**. Si no hay reglas aplicables se detiene el intérprete.
 4. Reset: Vacía la lista de reglas aplicables y vuelve al paso 1.
- Si la memoria de trabajo tiene los siguientes hechos iniciales:

Diametro = 0.4 cm, forma = redonda, Numsemillas = 1, color = rojo

<i>Ciclo de</i>	<i>Reglas</i>	<i>regla</i>	<i>Hecho derivado</i>
<i>ejecución</i>	<i>aplicables</i>	<i>seleccionada</i>	

1	3, 4	3	claseFruta = árbol
2	3, 4	4	claseSemilla = hueso
3	3, 4, 10	10	Fruta = cereza
4	3, 4, 10	—	

Encadenamiento regresivo

- Se comienza por un objetivo (conclusión que se desea probar) y decide si los hechos soportan el objetivo.
 - Se comienza con una **base de hechos**: (), que suele estar vacía y
 - **Una lista de objetivos** para la que el sistema intenta derivar hechos. Los objetivos se ordenan de forma que sean los más fácilmente alcanzables primero. Por ejemplo, en el problema de identificar fruta el objetivo es dar valor al parámetro “Fruta”. **Objetivos**: (Fruta).
 - El encadenamiento regresivo utiliza la lista de objetivos para coordinar su búsqueda a través de la base de reglas.

Pasos del intérprete con encadenamiento regresivo

1. Forma una **pila** con todos los **objetivos iniciales**.
2. Reunir todas las reglas capaces de satisfacer el primer objetivo.
3. Para cada una de estas reglas, examinar sus premisas:
 - a) Si las **premisas son satisfechas**, entonces se ejecuta esta regla para derivar sus conclusiones. **Elimina el objetivo de la pila y vuelve al paso 2.**
 - b) Si una de las premisas no se cumple, busca las reglas que pueden derivar esta **premisa**. Si se encuentra alguna regla, entonces se considera la premisa como subobjetivo, se coloca éste al principio de la pila, y se va al paso 2.
 - c) Si el paso b no puede encontrar una regla que derive el valor especificado para el objetivo en curso, **entonces preguntar al usuario** por el valor del parámetro, y añade éste a la memoria de trabajo. Si este valor satisface la premisa en curso, continúa con la siguiente premisa de esta regla. Si la premisa en curso no queda satisfecha por el valor continúa con la siguiente regla.
4. **Si todas las reglas que pueden satisfacer el objetivo actual se han intentado y han fallado, entonces el objetivo en curso permanece indeterminado. Saca éste de la pila y vuelve al paso 2. Si la pila de objetivos está vacía, el intérprete se detiene.**

Ejemplo de encadenamiento regresivo I

- Supongamos que queremos examinar una cereza. La traza de ejecución de las reglas para ver si son capaces de derivar cereza como valor de fruta se como sigue:
 - Paso 1. **Objetivos: (Fruta)**
 - Paso 2. La **lista de reglas** que pueden satisfacer este objetivo son: **1, 6,7, 8, 9, 10, 11, 12, y 13.**
 - Paso 3.
 - Se considera regla 1: La primera premisa (Forma= alargada) no se encuentra en la memoria de trabajo. No hay reglas que deriven éste valor así que el intérprete pregunta por este valor:
 - **¿Cuál es el valor de Forma? redondo.**
 - **Memoria de Trabajo: ((Forma = redondo))**
 - Se considera regla 6: La primera premisa de esta regla es (claseFruta = planta), y no se encuentra la memoria de trabajo. Reglas 2 y 3 pueden derivar éste valor, así que añadimos claseFruta en la lista de objetivos:
 - **Objetivos: (claseFruta, Fruta)**

Objetivos: (Fruta)**Hechos: Forma= redondo****¿Cuál es el valor de forma?: Redondo**

REGLAS PARA IDENTIFICAR FRUTA

Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana

Regla 2: SI Forma = redonda u ovalada
Diametro > 1.6 cm
ENTONCES claseFruta = planta

Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol

Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso

Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple

Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía

Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón

Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricocoque

Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja

Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza

Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón

Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana

Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Objetivos: (ClaseFruta , Fruta)

¿Cuál es el valor de Diametro?:0.4

Hechos: Forma= redondo , Diametro = 0.4

REGLAS PARA IDENTIFICAR FRUTA

Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana

Regla 2: SI Forma = redonda u ovalada
Diametro > 1.6 cm
ENTONCES claseFruta = planta

Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol

Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso

Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple

Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía

Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón

Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque

Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja

Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza

Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón

Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana

Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Ejemplo de encadenamiento regresivo II

- Examinamos regla 2, la primera premisa (Forma redondo o alargado) es satisfecha puesto que el valor de “Forma” es redondo. Se continúa con la siguiente premisa, puesto que no existe un valor de diámetro ni se puede derivar de otras reglas se pregunta al usuario:
 - ¿Cuál es el valor del diametro? 0.4
 - Memoria de Trabajo:
`((Forma = redondo) (Diámetro = 0.4))`
- La regla 2 falla. El intérprete lo intenta con la **regla 3**. Ambas premisas se cumplen, por lo que se deriva que claseFruta = árbol
 - Memoria de Trabajo:
`((Forma = redondo) (Diámetro = 0.4)
(claseFruta = árbol))`
- Como se ha encontrado un valor para el el objetivo claseFruta se elimina éste de la lista de objetivos. Se vuelve al objetivo Fruta y a la regla 6. Falla la segunda premisa claseFruta=planta. Lo mismo ocurre con la regla 7.

Objetivos: (Fruta)

Hechos: Forma= redondo, Diametro = 0.4, claseFruta = arbol

REGLAS PARA IDENTIFICAR FRUTA

- Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana
- Regla 2: SI Forma = redonda u ovalada
Diametro > 1.6 cm
ENTONCES claseFruta = planta
- Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol
- Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso
- Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple
- Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía
- Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón
- Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque
- Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja
- Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza
- Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón
- Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana
- Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Ejemplo de encadenamiento regresivo III

- La regla 8 tiene su primera premisa satisfecha (`claseFruta = árbol`), la siguiente premisa color no está ni se puede derivar:
 - ¿Cuál es el valor del color? rojo
 - Memoria de Trabajo:


```
((Forma = redondo) (Diámetro = 0.4)
  (claseFruta = árbol) (color rojo))
```
- Fallan reglas 8 y 9 porque sus premisas “color” no son rojo. La regla 10 cumple las 2 primeras premisas (`(claseFruta = árbol)` y `(color = rojo)`). No hay valor para la tercera premisa (`claseSemilla = hueso`) ni hay reglas que puedan derivarlo:
 - ¿Cuál es el valor de claseSemilla? hueso
 - Memoria de Trabajo:


```
((Forma = redondo) (Diámetro = 0.4) (claseFruta =
  árbol) (color rojo) (claseSemilla = hueso))
```
- La regla 10 es satisfecha completamente, se deriva el valor de fruta y queda la memoria de trabajo con el valor de fruta. La pila de objetivos se vacía:
 - Memoria de Trabajo:


```
((Forma = redondo) (Diámetro = 0.4) (claseFruta =
  árbol) (color rojo) (claseSemilla = hueso) (Fruta = cereza))
```
 - Objetivos: ()

Objetivos: (Fruta)

Hechos: Forma= redondo, Diametro = 0.4, claseFruta = arbol ,color = rojo
,claseSemilla = hueso

REGLAS PARA IDENTIFICAR FRUTA

- Regla 1: SI Forma = alargada y
Color = verde o amarillo
ENTONCES Fruta = banana
- Regla 2: SI Forma = redonda u ovalada
Diametro > 1.6 cm
ENTONCES claseFruta = planta
- Regla 3: SI Forma = redonda y
Diametro < 1.6 cm
ENTONCES claseFruta = árbol
- Regla 4: SI numSemillas = 1
ENTONCES claseSemilla = hueso
- Regla 5: SI numSemillas > 1
ENTONCES claseSemilla = multiple
- Regla 6: SI claseFruta = planta y
Color = verde
ENTONCES Fruta = sandía
- Regla 7: SI Forma = planta y
Color = amarillo
ENTONCES Fruta = melón
- Regla 8: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = albaricoque
- Regla 9: SI claseFruta = árbol y
Color = naranja y
claseSemilla = multiple
ENTONCES Fruta = naranja
- Regla 10: SI claseFruta = árbol y
Color = rojo y
claseSemilla = hueso
ENTONCES Fruta = cereza
- Regla 11: SI claseFruta = árbol y
Color = naranja y
claseSemilla = hueso
ENTONCES Fruta = melocotón
- Regla 12: SI claseFruta = árbol y
Color = rojo o amarillo o verde y
claseSemilla = múltiple
ENTONCES Fruta = manzana
- Regla 13: SI claseFruta = árbol y
Color = morado y
claseSemilla = hueso
ENTONCES Fruta = ciruela

Distinción Razonamiento / Encadenamiento

- Distinción entre razonamiento y encadenamiento progresivo y regresivo:
 - **Razonamiento bottom-up** o progresivo: De los hechos a los objetivos
 - **Encadenamiento progresivo**: comparar la parte derecha (SI) de las reglas con los datos de la memoria de trabajo, y ejecutar la parte derecha (ENTONCES) de las reglas satisfechas.
 - **Razonamiento top-down** o regresivo: De los objetivos a los hechos:
 - **Encadenamiento regresivo**: Se comienza por un objetivo y se busca las reglas capaces de satisfacer el objetivo en su parte derecha. Para cada una de estas reglas se miran sus precondiciones.
- Un sistema de producción como CLIPS tienen un mecanismo de encadenamiento hacia adelante. Sin embargo, en CLIPS se puede hacer razonamiento regresivo si se hace un control explícito del encadenamiento:
 - El encadenamiento implementa el razonamiento
 - La estrategia de razonamiento puede controlar el encadenamiento

Estrategias de control

- En cada ciclo de interpretación puede haber más de una instancia de regla candidata a la ejecución.
- Hay dos aproximaciones generales al control de los sistemas basados en reglas
 - **Control global:** Control independiente del dominio de aplicación.
 - Estrategias implementadas en el intérprete
 - No son modificables por el programador
 - **Control local:** Control dependiente del dominio de aplicación.
 - Reglas especiales que permiten razonar sobre el control: METAREGLAS.
 - El programador escribe reglas explícitas para controlar el sistema.

Control Global

- Determina que reglas participan en el proceso de reconocimiento de patrones y como se elegirá entre las instancias de regla en el caso de que exista más de una candidata.
- Dos mecanismos:
 - **Estrategias de resolución del conjunto conflicto:** Estrategias para la selección de una regla del conjunto conflicto para ser disparada en cada ciclo.
 - **Proceso de filtrado:** Decide que reglas y con que datos se intenta realizar el proceso de reconocimiento de patrones.
 - En CLIPS se comparan todas las reglas con todos los hechos de la memoria de trabajo. Pero se pueden definir módulos.
 - En KEE, es posible agrupar las reglas en clases de forma que las inferencias se realicen solo con instancias de una cierta clase (de reglas).

Resolución de conflictos

- Los criterios de selección persiguen que el sistema sea sensible y estable *(Brownston y col. 1985, Programming Expert Systems in OPS5, capítulo 7)*
 - Sensible: Se responda a cambios reflejados en la memoria de trabajo
 - Estable: Haya continuidad en la línea de razonamiento.
- Los mecanismos de resolución de conflicto son muy diversos, pero los siguientes **criterios** de selección son muy populares
 - **Refracción(refraction)**: Una regla no debe poderse disparar más de una vez con los mismo hechos.
 - **Novedad (recency)**: Las instancias de reglas que utilizan hecho más recientes son preferidas a las que utilizan hechos más viejos.
 - **Especificidad**: Instancias derivadas de reglas más específicas (con mayor número de condiciones, es decir más difíciles de cumplir) son preferidas.
 - **Prioridades** asociadas a las reglas.

Ventajas y desventajas de los LBR

Ventajas

- Modularidad: Los lenguajes basados en reglas son muy modulares.
 - Cada regla es una unidad del conocimiento que puede ser añadida, modificada o eliminada independientemente del resto de las reglas.
 - Se puede desarrollar una pequeña porción del sistema, comprobar su correcto funcionamiento y añadirla al resto de la base de conocimiento.
- Uniformidad:
 - Todo el conocimiento es expresado de la misma forma.
- Naturalidad:
 - Las reglas son la forma natural de expresar el conocimiento en cualquier dominio de aplicación.
- Explicación:
 - La traza de ejecución permite mostrar el proceso de razonamiento
 - En CLIPS

(watch rules)

Ventajas y desventajas de los LBR

Desventajas

- Ineficiencia: La ejecución del proceso de reconocimiento de patrones es muy ineficiente.
- Opacidad: Es difícil examinar una base de conocimiento y determinar que acciones van a ocurrir.
 - La división del conocimiento en reglas hace que cada regla individual sea fácilmente tratable, pero se pierde la visión global.
- Dificultad en cubrir todo el conocimiento:
 - Aplicaciones como el control de tráfico aéreo implicarían una cantidad de reglas que no serían manejables.

Sistema de Ayuda toma de decisiones

PARTE 2

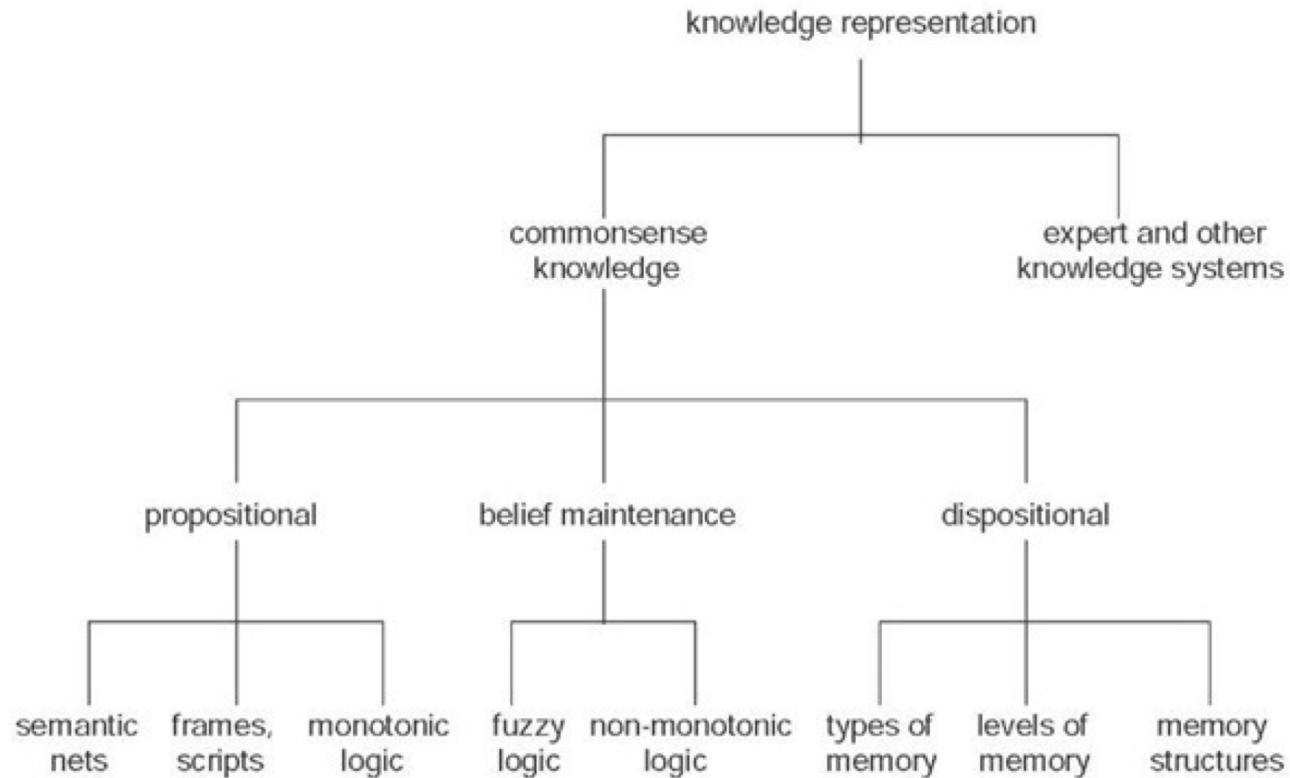
Luis Fernando Castillo Ossa

Sistemas Inteligentes 1

Representación Conocimiento

Representación de Conocimiento (Knowledge representation (KR)): Es el estudio de Cómo el conocimiento acerca del “mundo” puede representarse y que inferencias pueden deducirse de este conocimiento: Research at the Computational Intelligence Research Laboratory ([CIRL](#)) [at the University of Oregon](#)

Representación Conocimiento



http://lh3.ggpht.com/_1wtadqGaaPs/TH_OWQGWXHI/AAAAAAAAAXFA/

Inferencia con Lógica Predicados

Implicaciones Notables

- | | |
|--------------------------|-------------------------|
| 1. Modus ponens | 8. Ley de transposición |
| 2. Modus tollens | 9. Ley de traslación |
| 3. Modus tollendo ponens | 10. Leyes de Morgan |
| 4. Ley conjuntiva | 11. Dilema constructivo |
| 5. Ley simplificativa | 12. Dilema destructivo |
| 6. Ley aditiva | 13. Ley del condicional |
| 7. Silogismo condicional | . |

1. Modus ponens

$$\begin{array}{l} 1. \quad p \rightarrow q \\ 2. \quad p \\ \hline \therefore q \end{array}$$

Ejemplo 1.4

- "Si llueve, las calles se mojan"
- "Está lloviendo"

Luego:

- "Las calles se están mojando"

Herramienta Computacional Inferencias (Prolog) Básica en Lógica Primer Orden

```
GNU Prolog console
File Edit Terminal Prolog
yes
| ?- hermanos(X,Y)
.
X = pedro
Y = pedro ? ;
X = pedro
Y = paco ? ;
X = paco
Y = pedro ? ;
X = paco
Y = paco ? ;
X = paty
Y = paty ? ;
X = hugo
Y = hugo ? ;
X = hugo
Y = fer ? ;
X = fer
Y = hugo ? ;
X = fer
Y = fer ? ;
no
| ?-
```

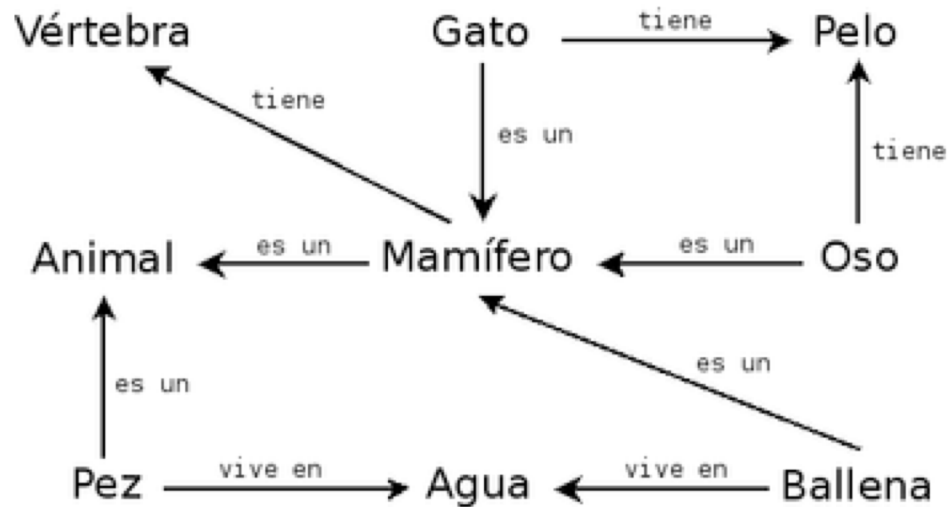
```
progenitor(pedro, teresa).
progenitor(maria, teresa).
progenitor(maria, elena).
progenitor(teresa, jorge).
progenitor(teresa, raquel).
progenitor(raquel, miguel).
```

```
hombre(pedro).
mujer(maria).
mujer(elena).
mujer(teresa).
hombre(jorge).
mujer(raquel).
hombre(miguel).
```

```
hermana(X, W) :-
    progenitor(Z, X), progenitor(Z, W),
    mujer(X).
```

Algoritmo de
Unificación

Red Semántica



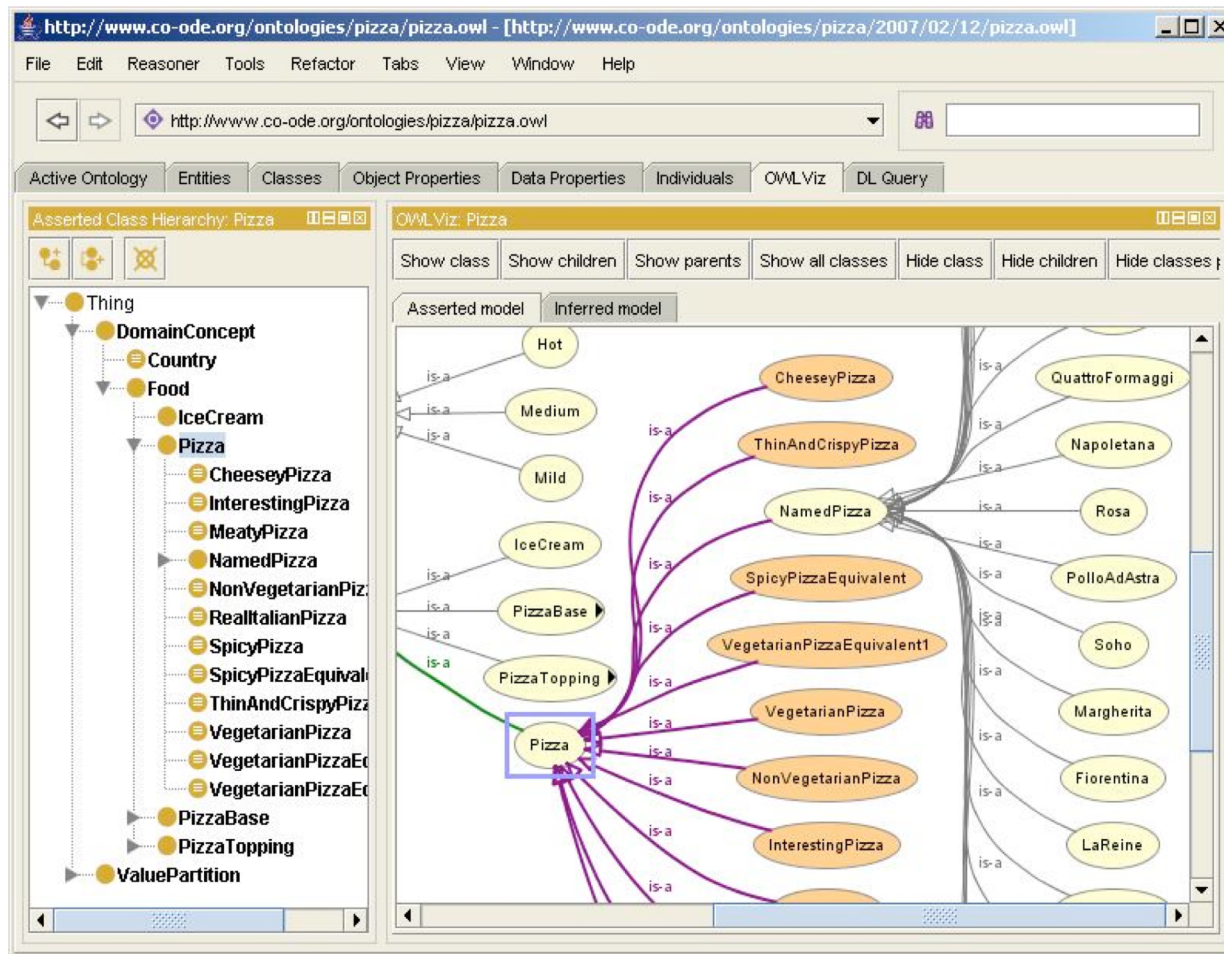
Una red semántica o esquema de representación en Red es una forma de representación de conocimiento lingüístico en la que los conceptos y sus interrelaciones se representan mediante un grafo.

Redes Semánticas

- Las redes están presentes en la mayoría de las técnicas de representación
 - Los **grafos** están incluidos en las redes, y estos son **la estructura abstracta más común en toda la computación**.
 - Ejemplos de redes
 - Redes taxonómicas
 - clasificación conceptos
 - jerarquías de herencia
 - reutilización de código/ herencia de propiedades (inferencia)
 - Redes bayesianas
 - Sistemas de diagnóstico basado en métodos probabilísticos, en el que la red representa las dependencias de las variables.
 - Redes de Petri
 - Representación de procesos concurrentes.

Ontologías (Editor de Ontologías)

Open Source



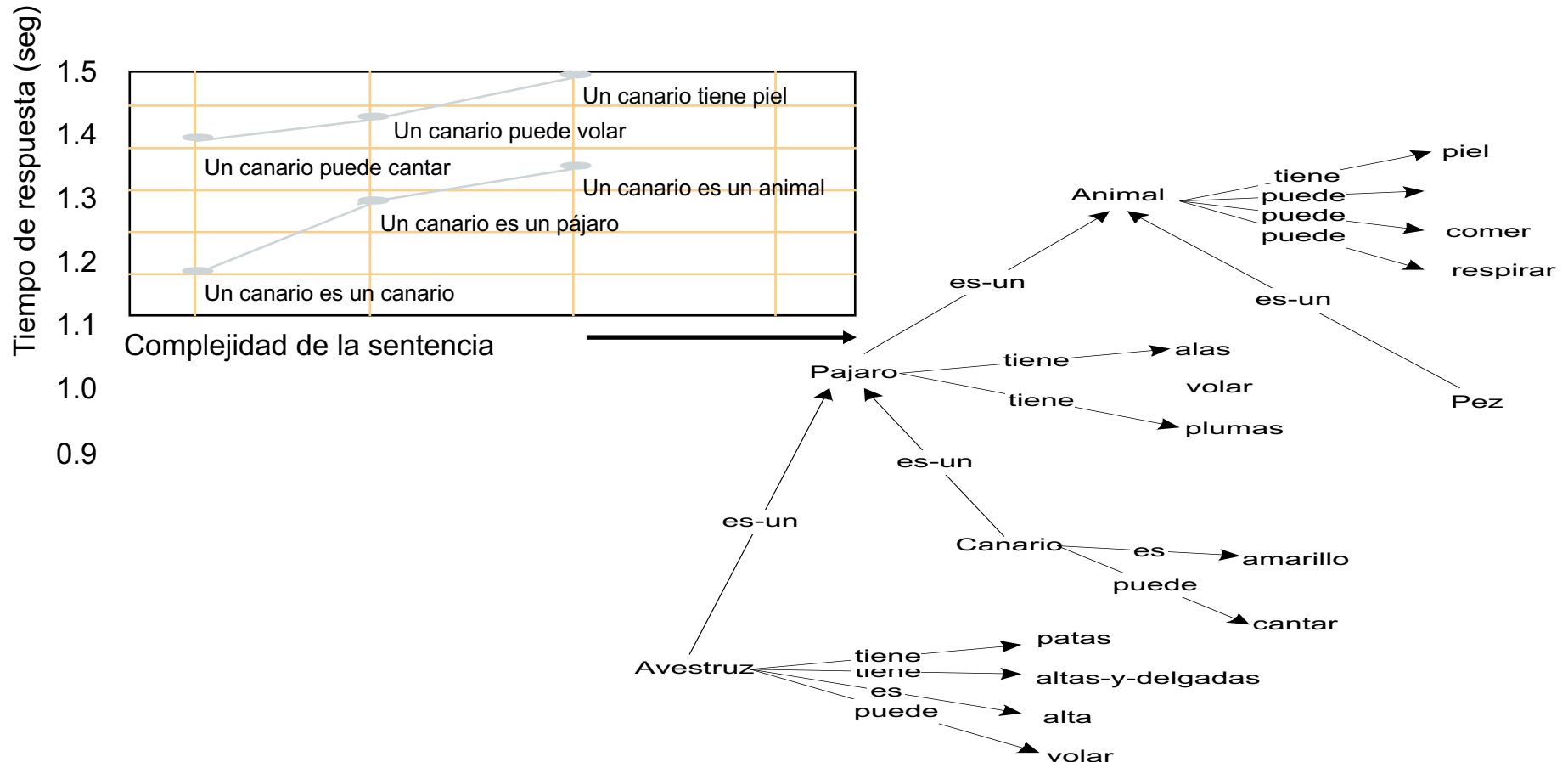
Protégé is a core component of The National Center for Biomedical Ontology

Ontologías

W3C,"una ontología define los términos a utilizar para describir y representar un área del conocimiento. Las ontologías son utilizadas por las personas, las bases de datos, y las aplicaciones que necesitan compartir un dominio de información

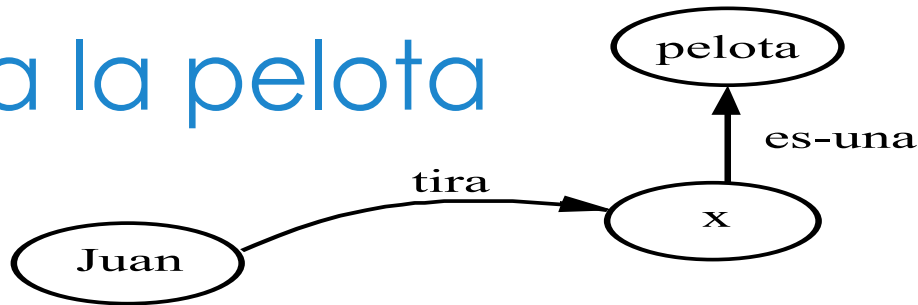
Resultado

- Red semántica desarrollada por Collins y Quilian (1969) en su investigación sobre almacenamiento de información humana y tiempos de respuesta



Juan tira la pelota

- Red:



- Lógica:

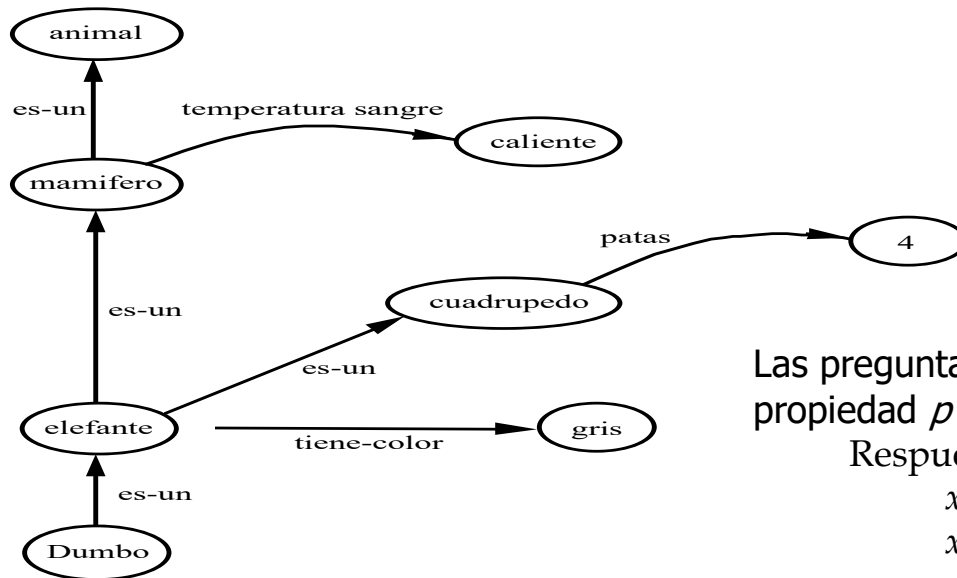
- $\exists x. \text{Tira}(\text{Juan}, x) \wedge \text{Pelota}(x)$

- Problema:

- Nos gustaría aportar información extra sobre el evento "Tira", tal como el tiempo, lugar, o intensidad.

Inferencia: Herencia de propiedades

- Herencia es el algoritmo de inferencia más común en los sistemas de redes semánticas.
- A través de relaciones es-un e instancia



Las preguntas son de la forma: “cuál es el valor de la propiedad p para el nodo x ”

Respuesta cuando

x tiene la propiedad p

x es-un y e y tiene la propiedad p

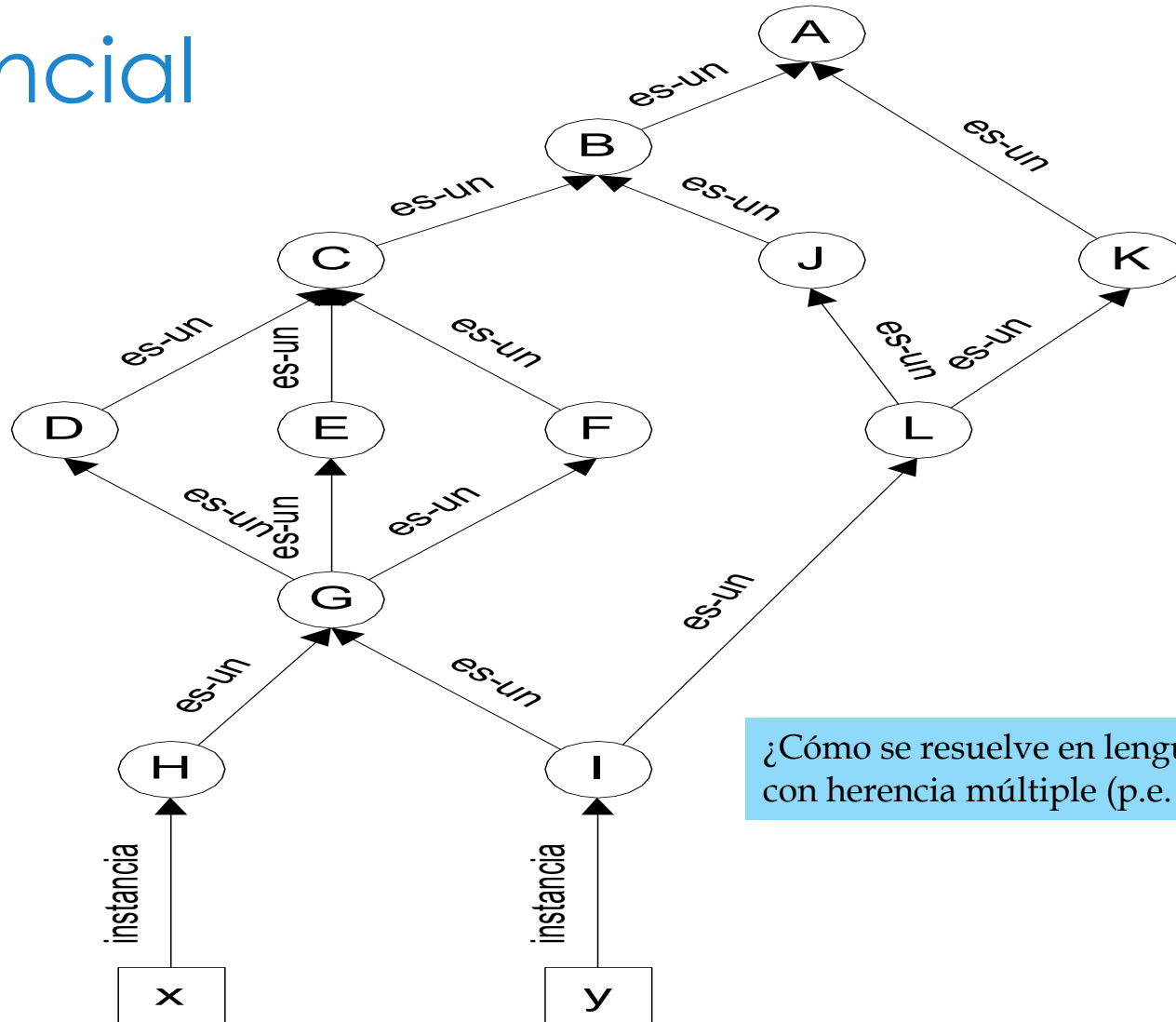
x es-un z_1 , z_1 es-un z_2 , ..., z_{n-1} es-un z_n , y z_n tiene la propiedad p

En otro caso, la respuesta está indefinida

Distancia inferencial

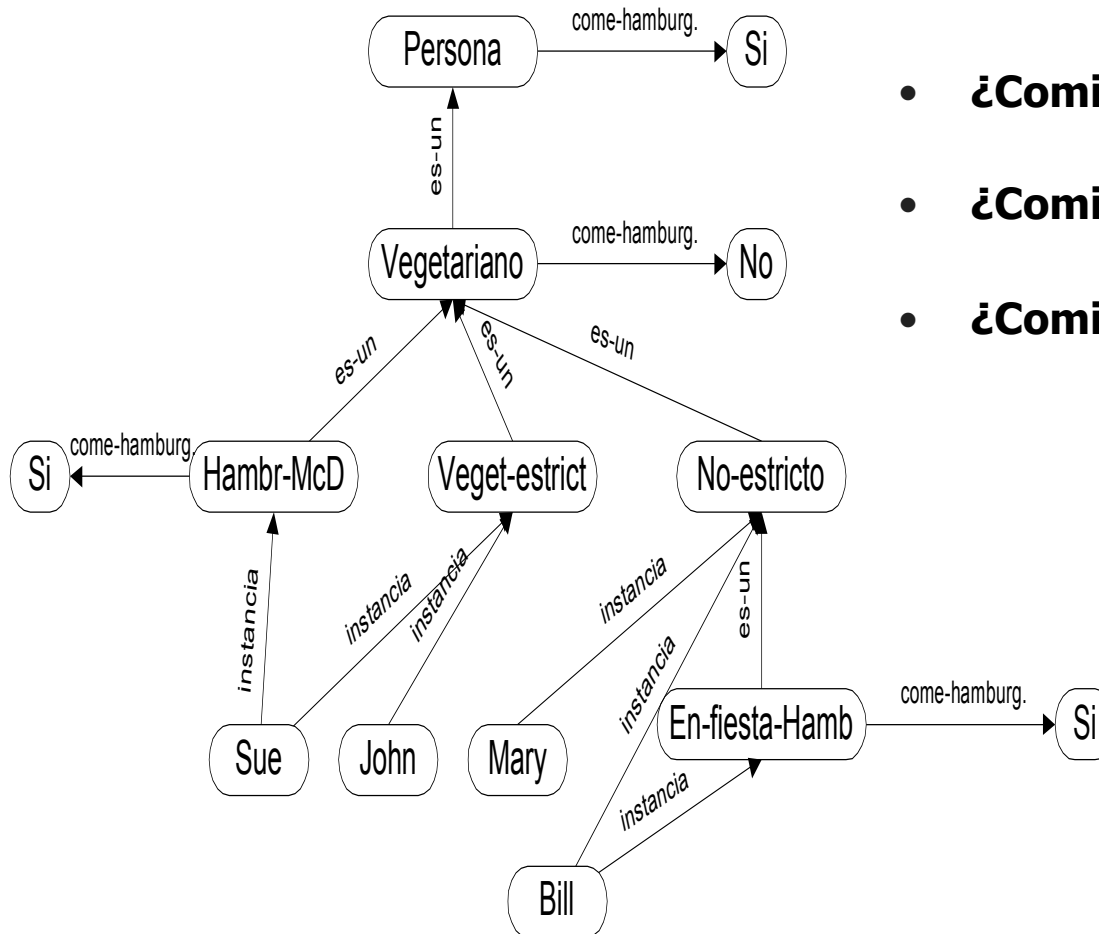
- Touretzky 1986
“The mathematics of inheritance systems”
 - Si α y β son antecesoros de x , y α es antecesor de β , entonces β es inferencialmente mas cercano a x que α .
 - Si α y β tienen ambos un valor para la propiedad p , x heredará su valor para la propiedad p de β (e ignorará completamente el valor para p de α)
 - Si α no es un antecesor de β y β no es un antecesor de α , entonces α y β están en conflicto en lo que se refiere a la herencia de x : si α y β tienen diferentes valores para la propiedad p , entonces x no tendrá un valor para la propiedad p
 - Los nodos que no son antecesoros de x son completamente irrelevantes en lo que se refiere a la herencia de x .

Ejemplo distancia inferencial



¿Cómo se resuelve en lenguajes con herencia múltiple (p.e. CLOS)?

Red semántica del ejemplo



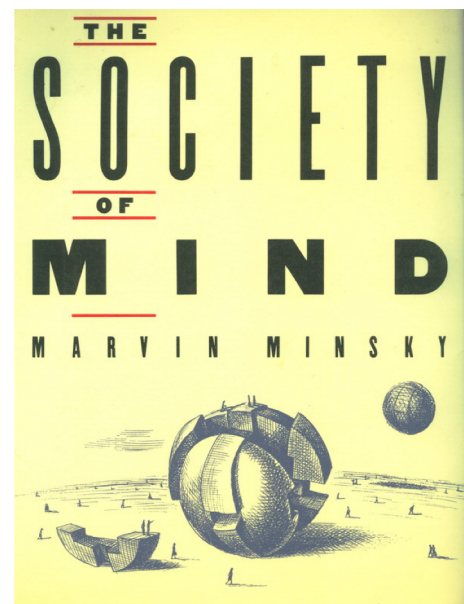
- **¿Comió John alguna hamburguesa?**
- **¿Comió Sue alguna hamburguesa?**
- **¿Comió Mary alguna hamburguesa?**
- **¿Comió Bill alguna hamburguesa?**

Frames (marcos)

- Minsky MIT 1975

Los sistemas basados en frames fueron propuestos originariamente por Marvin del MIT, en un artículo:

- "A Framework for Representing Knowledge"
- Su trabajo está relacionado con el del psicólogo Jean Piaget.
- Minsky estaba interesado en la ayuda que un sistema de frames podía proporcionar para controlar el razonamiento en un sistema de percepción en el campo de visión por computador o en la comprensión del lenguaje natural



“

No computer has ever been designed that is ever aware of what it's doing; but most of the time, we aren't either.

—Marvin Minsky

¿Que son los frames?

- Frame de una "habitación"
 - El frame de una "habitación" debería contener componentes como puerta, suelo, paredes, y techo.
 - El frame de la "habitación" debería incluir también cierta información esperada: "las paredes de la habitación son planas y se juntan en ángulos rectos".
 - Ciertos tipos de puertas dependen de ciertos tipos de habitaciones. Incluso antes de entrar en una habitación podríamos decir si se entra en un armario, una habitación o se sale al exterior.
 - Podrían crearse subconceptos de "habitación", p.e. frames para "comedor", "dormitorio" o "baño".
 - En el frame del "baño", esperaríamos no encontrar una puerta de cristal.

Los frames después de Minsky

- Los sistemas actuales reflejan el espíritu anterior pero su forma ha evolucionado considerablemente
 - Objetivo adicional: **manipular conocimiento difícil de manipular usando cálculo de predicados**
 - valores por defecto,
 - excepciones e
 - información redundante o incompleta
 - Se quiere una estructura flexible, con capacidad para evolucionar
- **Nos interesa su aspecto computacional. Ideas clave:**
 - **Jerarquía y herencia** son los conceptos básicos asociados a los frames
 - Uso de conocimiento por **defecto**
 - Posibilidad de asociar procedimientos para completar capacidades
 - **Métodos y demonios.**

Aspectos de interés para la programación

- Los frames son prototipos:
 - Representa información por defecto que caracteriza una familia de entidades.
 - Referencia para comparar objetos a ser reconocidos, analizados, clasificados.

Herencia
Clases
Instancias

- Generalmente están organizados en una estructura a tres niveles (objeto, atributo, valor):

Frame (objeto)	slot (atributo)	facet (valor)
Frame: entidad Slot: propiedades de la entidad Facet: características descriptivas o de comportamiento del slot		

Conclusiones sobre frames

■ Ventajas

- Incluye provisión para la asociación de conocimiento procedural con el conocimiento declarativo almacenado como valores de slots (esto permite permite expresar conocimiento más allá del puramente lógico)
- Estructura de control flexible; el comportamiento de FRL es fácil de extender

■ Desventajas

- Conduce a **una frágil teoría de la representación**; hace difícil la especificación de la "**corrección**" del sistema.



Sistemas Inteligentes

lección Redes y Frames

Sistemas Inteligentes 1

